```c
/*
 * @file    Ejercicio_8_3.c
 * @brief   Realiza la generación de varias formas de ondas desde tres buffers
 *          específico en la AHB SRAM a través del módulo DAC, de manera indefinida
 *          mediante LLIs multiples de forma circular. Monitoreando el estado
 *          de la tranferencia mediante interrupción del GPDMA.
 * @details Configuración del Periférico DAC para Transferencia M2P con LLI
 *          Multiple e ISR vía GPDMA.
 * @author  Ing. E. Migliore
 * @date    30/04/2026
 * @version 2.0
 */

/* Directivas de Inclusión */
#include "LPC17xx.h"
#include "lpc17xx_gpdma.h"

/* Definición de Constantes y Macros */
#define DAC_BUFFER0_START 0x2007C000
#define DAC_BUFFER1_START 0x2007D000
#define DAC_BUFFER2_START 0x2007E000
#define NUM_WAVE_SAMPLES 1024

/* Definición de Variables Globales */
//WaveForm0
volatile uint32_t *dac_samples0 = (volatile uint32_t *)DAC_BUFFER0_START;
//WaveForm1
volatile uint32_t *dac_samples1 = (volatile uint32_t *)DAC_BUFFER1_START;
//WaveForm2
volatile uint32_t *dac_samples2 = (volatile uint32_t *)DAC_BUFFER2_START;

volatile uint32_t DMA_CH0_TC = 0;   //Flag de Cuenta Terminal para Canal 0 de DMA
volatile uint32_t DMA_CH0_Err = 0;  //Flag de Error Terminal para Canal 0 de DMA
volatile bool DAC_FLAG = true;

/* Prototipos de Funciones */
void cfgDMA(void);
void loadDACwaves(void);

/* Función Principal  */
int main(void)
{
    cfgDAC();
    cfgDMA();

    //Trigger por software
    while(1) if(DAC_FLAG) { loadDACwaves(); DAC_FLAG = false; }

    return 0;
}

/* Definiciones de Funciones */
void loadDACwaves(void)
{
    //Inicialización de Envió de Datos por Canal 0 de DMA
    GPDMA_ChannelCmd(LPC_GPDMACH0, ENABLE);
    //Habilitación de Interrupción por DMA
    NVIC_EnableIRQ(DMA_IRQn);
    return;
}

void cfgDMA(void)
{
    //Estructura para Lista Enlazada 0 para canal 0 del DMA
    static GPDMA_LLI_Type cfgDAC_LLI0_CH0;
    //Estructura para Lista Enlazada 1 para canal 0 del DMA
    static GPDMA_LLI_Type cfgDAC_LLI1_CH0;
    //Estructura para Lista Enlazada 2 para canal 0 del DMA
    static GPDMA_LLI_Type cfgDAC_LLI2_CH0;
```

```
70      //Estructura de Configuración para canal 0 del DMA
71      GPDMA_Channel_CFG_Type cfgDAC_DMA_CH0;
72
73      //Deshabilitación de Interrupción por DMA
74      NVIC_DisableIRQ(DMA_IRQn);
75      //Inicialización del Controlador GPDMA
76      GPDMA_Init();
77
78      //Dirección de Fuente de Datos (Memoria)
79      cfgDAC_LLI0_CH0.SrcAddr = (uint32_t)dac_samples0;
80      //Dirección de Destino de Datos (Periferico)
81      cfgDAC_LLI0_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR);
82      //Próxima Dirección de Destino de Datos: LLI Repetitiva
83      cfgDAC_LLI0_CH0.NextLLI = (uint32_t)&cfgDAC_LLI1_CH0;
84      //Tamaño de la Transferencia de la LLI0
85      cfgDAC_LLI0_CH0.Control = (NUM_WAVE_SAMPLES<<0)
86      //Ancho de la Transferencia de Fuente (32-bit)
87                              |(2<<18)
88      //Ancho de la Transferencia de Destino (32-bit)
89                              |(2<<21)
90      //SI=1: Con Incremento de la Dirección de Fuente
91                              |(1<<26)
92      //DI=0: Sin Incremento de la Dirección de Destino
93                              & ~(1 << 27);
94      //Dirección de Fuente de Datos (Memoria)
95      cfgDAC_LLI1_CH0.SrcAddr = (uint32_t)dac_samples1;
96      //Dirección de Destino de Datos (Periferico)
97      cfgDAC_LLI1_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR);
98      //Próxima Dirección de Destino de Datos: LLI Repetitiva
99      cfgDAC_LLI1_CH0.NextLLI = (uint32_t)&cfgDAC_LLI2_CH0;
100     //Tamaño de la Transferencia de la LLI1
101     cfgDAC_LLI1_CH0.Control = (NUM_WAVE_SAMPLES<<0)
102     //Ancho de la Transferencia de Fuente (32-bit)
103                             |(2<<18)
104     //Ancho de la Transferencia de Destino (32-bit)
105                             |(2<<21)
106     //SI=1: Con Incremento de la Dirección de Fuente
107                             |(1<<26)
108     //DI=0: Sin Incremento de la Dirección de Destino
109                             & ~(1 << 27);
110     //Dirección de Fuente de Datos (Memoria)
111     cfgDAC_LLI2_CH0.SrcAddr = (uint32_t)dac_samples2;
112     //Dirección de Destino de Datos (Periferico)
113     cfgDAC_LLI2_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR);
114     //Próxima Dirección de Destino de Datos: LLI Repetitiva
115     cfgDAC_LLI2_CH0.NextLLI = (uint32_t)&cfgDAC_LLI0_CH0;
116     //Tamaño de la Transferencia de la LLI2
117     cfgDAC_LLI2_CH0.Control = (NUM_WAVE_SAMPLES<<0)
118     //Ancho de la Transferencia de Fuente (32-bit)
119                             |(2<<18)
120     //Ancho de la Transferencia de Destino (32-bit)
121                             |(2<<21)
122     //SI=1: Con Incremento de la Dirección de Fuente
123                             |(1<<26)
124     //DI=0: Sin Incremento de la Dirección de Destino
125                             & ~(1 << 27);
126
127     //Selección del Canal de Transferencia
128     cfgDAC_DMA_CH0.ChannelNum = 0;
129     //Periférico de Fuente de Datos (Sin Uso)
130     cfgDAC_DMA_CH0.SrcConn = 0;
131     //Periférico de Destino de Datos (DAC)
132     cfgDAC_DMA_CH0.DstConn = GPDMA_CONN_DAC;
133     //Dirección de Fuente de Datos en Memoria
134     cfgDAC_DMA_CH0.SrcMemAddr = cfgDAC_LLI0_CH0.SrcAddr;
135     //Dirección de Destino de Datos en Memoria (Sin Uso)
136     cfgDAC_DMA_CH0.DstMemAddr = 0;
137     //Tipo de Transferencia (Memoria a Periférico)
138     cfgDAC_DMA_CH0.TransferType = GPDMA_TRANSFERTYPE_M2P;
```

```
    //Tamaño de Transferencia de Datos
    cfgDAC_DMA_CH0.TransferSize = NUM_WAVE_SAMPLES;
    //Ancho de Transferencia de Datos (Sin Uso)
    cfgDAC_DMA_CH0.TransferWidth = 0;
    //Lista Enlazada Inicial
    cfgDAC_DMA_CH0.DMALLI = (uint32_t)&cfgDAC_LLI0_CH0;
    //Configuración del Canal 0 de DMA
    GPDMA_Setup(&cfgDAC_DMA_CH0);

    return;
}

void DMA_IRQHandler (void){
    //Verifica Interrupción por Canal 0 de DMA
    if(GPDMA_IntGetStatus(GPDMA_STAT_INT, 0)){
        //Verifica Interrupción por Cuenta Terminal (1 List)
        if(GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 0)){

            //Ejecuta ISR: DMA_CH0_TC++;

            //Limpia Bandera de Interrupción por Cuenta Terminal
            GPDMA_ClearIntPending (GPDMA_STATCLR_INTTC, 0);
        }
        //Verifica Interrupción por Error Terminal
        if(GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 0)){

            //Ejecuta ISR: DMA_CH0_Err++;

            //Limpia Bandera de Interrupción por Error Terminal
            GPDMA_ClearIntPending (GPDMA_STATCLR_INTERR, 0);
        }
     }
     return;
}
```